

NIFTY100

ANALYTICS DASHBOARD

Analyst Guide

Streamlit Dashboard | FastAPI Service | Financial KPIs | Peer Analytics | Valuation | PDF Reports

Purpose: This guide explains how an analyst can run, navigate and validate the complete Nifty100 Analytics project without changing the production database.

Project Item	Value
Company universe	92 Nifty100 companies
Dashboard screens	Home, Profile, Screener, Peers, Trends, Sectors, Capital, Reports
REST API	FastAPI with 16 versioned endpoints
Automated tests	162 passing tests, 0 failures
Reporting	Company tearsheets, sector reports and portfolio summary

Prepared for: Manoj Seervi

Sprint 6 Final Documentation - 2026

1. Guide Overview

The Nifty100 Analytics Dashboard is a local financial analytics platform built with Python, SQLite, Pandas, Plotly, Streamlit and FastAPI. It combines company fundamentals, financial ratios, peer groups, market-cap history, NLP-generated observations, clustering and PDF reporting in one project.

Who should use this guide?

- Analysts who want to screen and compare Nifty100 companies.
- Project evaluators who need to verify dashboard, API and report deliverables.
- Developers who need repeatable run commands and troubleshooting steps.
- Interviewers or reviewers who want to understand the system workflow.

Recommended operating rule

Database safety: Treat nifty100.db as the working production database. Do not rebuild or overwrite it during routine testing. Keep a verified backup before ETL or schema operations.

Project architecture

Layer	Main responsibility	Key technology
Storage	Canonical company and financial data	SQLite
Analytics	Ratios, CAGR, cash-flow, clustering, percentiles	Pandas / NumPy / scikit-learn
Dashboard	Interactive analyst workflow	Streamlit / Plotly
API	Programmatic access to analytics	FastAPI
Reporting	Tearsheets and portfolio PDFs	Python PDF pipeline
Testing	ETL, KPI, DQ and API validation	pytest

2. Installation and Local Setup

Prerequisites

- Python 3.12 or a compatible Python 3.x environment.
- A project virtual environment named venv.
- The verified nifty100.db database in the project root.
- Dependencies installed from requirements.txt.

Windows setup commands

```
cd C:\Users\MANOJ\Desktop\project
venv\Scripts\activate.bat
python -m pip install -r requirements.txt
```

Verify the database

```
python -c "import sqlite3; c=sqlite3.connect('nifty100.db'); print('Companies:', c.execute('SELECT COUNT(*) FROM companies').fetchone()[0]); print('Ratios:', c.execute('SELECT COUNT(*) FROM financial_ratios').fetchone()[0])"
```

Expected company count is 92. The financial_ratios table should contain more than 1,100 rows in the final project state.

Create a working backup

```
copy /Y nifty100.db nifty100_working_backup.db
```

Important: Keep the backup outside automated loader logic. A test or ETL command should never silently replace the working database.

3. Starting the Dashboard and API

Start Streamlit

```
streamlit run src\dashboard\app.py --server.port 8501
```

Open the dashboard at:

```
http://localhost:8501/home
```

Start FastAPI

```
python -m uvicorn src.api.main:app --port 8000
```

Open Swagger documentation at:

```
http://127.0.0.1:8000/docs
```

Run both applications together

1. Open Terminal 1, activate venv and start Streamlit on port 8501.
2. Open Terminal 2, activate venv and start FastAPI on port 8000.
3. Use Terminal 3 for tests, curl commands and performance scripts.

Health verification

```
curl http://127.0.0.1:8000/api/v1/health
```

Pass condition: The API should return HTTP 200 with status=ok. Streamlit and FastAPI must remain on separate ports with no conflict.

4. Home and Company Profile Screens

Home Dashboard

The Home screen provides a quick entry point into the 92-company universe. It summarizes selected company KPIs and helps users move to deeper analysis screens.

- Use the company selector to choose a ticker.
- Review the latest ROE, D/E, revenue CAGR, composite quality score and market information.
- Use sector context to understand how the company fits into the reporting universe.

Company Profile

The Profile screen combines company details, financial history and analyst-friendly interpretation. It is the main screen for validating whether a company has consistent historical coverage.

- Select a ticker using the searchable company control.
- Review annual financial ratios excluding TTM where applicable.
- Inspect P&L, balance-sheet and cash-flow trends.
- Read pros and cons generated from available project data.
- Use peer and report links for deeper analysis.

Profile review checklist

Check	What to verify
Identity	Ticker and company name match the canonical companies table
Coverage	Historical financial data is available for roughly 10 years or more
Ratios	ROE, margins, leverage and cash-flow KPIs are numerically plausible
Narrative	Pros and cons are concise and supported by data
Performance	Screen should load within the project acceptance target

5. Stock Screener

The Screener is the primary analyst workflow. It joins the latest financial ratios with company, sector, market-cap and profit-and-loss data, then filters the 92-company universe.

Typical filter workflow

4. Open the Screener screen.
5. Start with no filters and confirm the complete company universe loads.
6. Set minimum ROE, maximum D/E, minimum free cash flow or valuation limits.
7. Optionally filter by reporting sector.
8. Review ranked companies and download the result as CSV.

Supported API-style filters

Filter	Meaning	Example
min_roe	Minimum return on equity	15
max_de	Maximum debt-to-equity	1
min_fcf	Minimum free cash flow	0
sector	Reporting sector	Information Technology
min_rev_cagr_5yr	Minimum five-year revenue CAGR	10
min_pat_cagr_5yr	Minimum five-year PAT CAGR	10
max_pe	Maximum P/E ratio	30

Example request

```
curl "http://127.0.0.1:8000/api/v1/screener?min_roe=15&max_de=1&min_fcf=0"
```

Financial-sector rule: Debt-to-equity behaves differently for banks and financial companies. The project intentionally avoids applying the same leverage interpretation to every sector.

6. Peer Comparison and Trend Analysis

Peer Comparison

Peer analysis groups comparable companies and uses percentile rankings to show relative strength. Benchmark companies are marked inside each peer group.

- Choose a peer group such as IT Services.
- Select up to three metrics for chart readability.
- Compare the selected company against peer averages and benchmark values.
- Use percentile ranks to avoid comparing raw values with different units.

Peer API examples

```
curl "http://127.0.0.1:8000/api/v1/peers/IT%20Services"  
curl "http://127.0.0.1:8000/api/v1/companies/INFY/peers/compare"
```

Trend Analysis

The Trends screen shows multi-year movement in profitability, leverage, efficiency, growth and cash-flow measures.

- Search using the Name (Ticker) format.
- Select only the metrics required for the current question.
- Use 10-year trends to distinguish one-year spikes from structural improvement.
- Treat missing or non-comparable historical values carefully.

Common trend questions

Question	Useful metric
Is profitability improving?	ROE, ROCE, NPM, OPM
Is leverage becoming risky?	Debt-to-equity, interest coverage
Is growth consistent?	Revenue CAGR, PAT CAGR, EPS CAGR
Is cash generation healthy?	CFO, FCF, FCF conversion
Is capital use efficient?	Asset turnover, capex intensity

7. Sector and Capital Allocation Analysis

Sector Analysis

The project uses 11 broad reporting sectors. The Sectors screen compares company profitability, valuation and scale within the selected sector.

- Select a broad sector from the dropdown.
- Use the bubble chart to compare valuation, profitability and market size.
- Review sector medians before judging an individual company.
- Use the IT alias only where supported; the canonical label is Information Technology.

Sector API examples

```
curl http://127.0.0.1:8000/api/v1/sectors
curl "http://127.0.0.1:8000/api/v1/sectors/IT/companies"
```

Capital Allocation

The Capital screen and generated reports summarize how companies deploy operating cash flows across capex, debt, dividends, buybacks and acquisitions.

- Compare cash from operations with free cash flow.
- Review capex intensity and whether the business is asset-light or capital-intensive.
- Check dividend and debt behavior over multiple years.
- Use the generated capital allocation report for portfolio-level interpretation.

8. Reports and PDF Tearsheets

Reports screen

The Reports screen exposes available annual-report links and project-generated PDF outputs. Annual report availability depends on the documents table.

Company tearsheets

A tearsheet is a compact company report containing key financial metrics, trends, peer context, valuation and qualitative observations.

9. Open the company Profile or Reports screen.
10. Select the required ticker.
11. Use the tearsheet download action, or call the API endpoint.
12. Open the PDF and verify there is no clipped text or chart overflow.

Tearsheet API

```
curl -o TCS_tearsheet.pdf http://127.0.0.1:8000/api/v1/companies/TCS/tearsheet
```

Other generated reports

Output	Purpose
reports/tearsheets/	Company-level PDF reports
reports/sectors/	Sector-level PDF summaries
reports/portfolio/portfolio_summary.pdf	Portfolio-wide company summary
reports/correlation_heatmap.png	Pearson correlation across 10 KPIs
reports/elbow_plot.png	KMeans elbow evidence for k=5

PDF acceptance: For sampled tearsheets, inspect every page for overflow, missing glyphs and clipped charts. File existence alone is not enough.

9. REST API Reference

The API uses the /api/v1 prefix and is documented interactively through Swagger. The generated OpenAPI specification is stored in docs/openapi.json.

Endpoint	Purpose
GET /api/v1/health	API and database health
GET /api/v1/companies	List/search companies
GET /api/v1/companies/{ticker}	Company profile
GET /api/v1/companies/{ticker}/pl	P&L history
GET /api/v1/companies/{ticker}/bs	Balance-sheet history
GET /api/v1/companies/{ticker}/cashflow	Cash-flow history
GET /api/v1/companies/{ticker}/ratios	Financial-ratio history
GET /api/v1/companies/{ticker}/tearsheet	Download tearsheet
GET /api/v1/screener	Run company screener
GET /api/v1/sectors	List sector summaries
GET /api/v1/sectors/{sector}/companies	Companies in sector
GET /api/v1/peers/{group_name}	Peer group percentiles
GET /api/v1/companies/{ticker}/peers/compare	Company peer comparison
GET /api/v1/market-cap/{ticker}	Market-cap and valuation history
GET /api/v1/portfolio/stats	Portfolio percentile statistics
GET /api/v1/companies/{ticker}/documents	Annual-report links

Useful curl examples

```
curl http://127.0.0.1:8000/api/v1/companies/TCS
curl http://127.0.0.1:8000/api/v1/companies/TCS/ratios
curl http://127.0.0.1:8000/api/v1/market-cap/TCS
curl http://127.0.0.1:8000/api/v1/portfolio/stats
```

10. Testing and Quality Assurance

The final test suite covers ETL loading, normalization, financial KPI logic, data-quality rules and API behavior.

Test area	Examples
ETL	Database creation, table presence, row availability, company IDs
Normalization	Year formats, TTM handling, ticker cleanup
KPI	ROE, ROCE, margins, CAGR, cash-flow quality and leverage
Data quality	14 rule-specific violation tests
API	Health, companies, screener, sectors, peers, valuation and documents

Run the full suite

```
python -m pytest tests --html=reports\pytest_report.html --self-contained-html
```

Final verified result: 162 tests passed with zero failures. A dependency deprecation warning may appear from the FastAPI test client; it does not count as a failed test.

Performance checks

```
python tests\performance\load_test.py
python tests\performance\dashboard_perf.py
python tests\performance\e2e_test.py
```

- Concurrent screener target: 10 calls complete within 10 seconds.
- Company Profile target: each of five sampled tickers loads within 3 seconds.
- E2E target: FastAPI and Streamlit run together without port conflict.

11. Troubleshooting

Problem	Likely cause	Action
no such column in dashboard	Wrong or outdated database restored	Stop servers and restore the verified working database backup
ModuleNotFoundError: src	Script started without project root on import path	Use python -m module.path or set PYTHONPATH=%CD%
API 403 during git push	Wrong cached GitHub account	Clear Git Credential Manager entry and authenticate with the repository owner
Swagger route missing	Router not included or file not saved	Inspect router.routes and app.openapi() ['paths']
Streamlit shows stale error	Cached query result	Stop app, run streamlit cache clear, then restart
Tests alter working data	Tests point at production DB	Use a temporary test database and keep a production backup
Tearsheet missing	Generation skipped or ticker filename mismatch	Compare company IDs against PDF stems and regenerate only the missing file

Safe recovery procedure

13. Stop Streamlit and FastAPI.
14. Copy the current database to a timestamped backup.
15. Inspect database schema and counts before restoring anything.
16. Restore only a verified compatible database.
17. Restart the dashboard and test Home, Profile and Screener first.
18. Do not run schema migrations against the restored database unless explicitly reviewed.

Golden rule: One change, one verification. Never combine database migration, test execution and dashboard debugging in the same uncontrolled step.

12. Analyst Workflow and Final Checklist

Recommended daily analyst workflow

19. Start Streamlit and FastAPI in separate terminals.
20. Verify API health and dashboard Home screen.
21. Use the Screener to create a candidate list.
22. Open company Profiles for financial-history review.
23. Compare candidates within peer groups and sectors.
24. Check valuation history and capital allocation.
25. Download tearsheets or CSV outputs for reporting.
26. Record assumptions and data limitations before making conclusions.

Final operational checklist

Item	Expected result
Database	92 companies and valid financial-ratio history
Dashboard	All eight screens open without red exceptions
API	Health endpoint returns HTTP 200
Tests	162 passed, 0 failed
Reports	Tearsheets, sector reports and portfolio summary available
Documentation	README, OpenAPI, Analyst Guide and acceptance checklist present
Backups	Verified working database backup stored safely

Interpretation disclaimer

This project is an educational analytics system. Financial ratios, model-generated clusters and qualitative observations should be reviewed with source documents and should not be treated as personalized investment advice.

End of Analyst Guide